

Get Docker

What is Docker?

Introduction

^

Get Docker Desktop

Develop with containers

Build and push your first image

What's next

Docker concepts

∨

Docker workshop

∨

Educational resources

Build and push your first image

 Ask Gordon

 Copy Markdown

 View Markdown



How to Build and Push Your First Docker |

Docker



Watch on

Explanation

Now that you've updated the [to-do list app](#), you're ready to create a container image for the application and share it on Docker Hub. To do so, you will need to do the following:

 [Edit this page](#)

 [Request changes](#)

Table of contents

Explanation

Container images

Docker Hub

Try it out

Sign in with your Docker acco...

Create an image repository

Build and push the image

Recap

Next steps

1. Sign in with your Docker account
2. Create an image repository on Docker Hub
3. Build the container image
4. Push the image to Docker Hub

Before you dive into the hands-on guide, the following are a few core concepts that you should be aware of.

Container images

If you're new to container images, think of them as a standardized package that contains everything needed to run an application, including its files, configuration, and dependencies. These packages can then be distributed and shared with others.

Docker Hub

To share your Docker images, you need a place to store them. This is where registries come in. While there are many registries, Docker Hub is the default and go-to registry for images. Docker Hub provides both a place for you to store your own images and to find images from others to either run or use as the bases for your own images.

When choosing base images, Docker Hub offers two categories of trusted, Docker-maintained images:

- [Docker Official Images \(DOI\)](#) – Curated images for popular software, following best practices and regularly updated.
- [Docker Hardened Images \(DHI\)](#) – Minimal, secure, production-ready images with near-zero CVEs, designed to reduce attack surface and simplify compliance. DHI images are free and open source under Apache 2.0.

In [Develop with containers](#), you used the following images that came from Docker Hub, each of which are [Docker Official Images](#):

- [node](#) - provides a Node environment and is used as the base of your development efforts. This image is also used as the base for the final application image.
- [mysql](#) - provides a MySQL database to store the to-do list items
- [phpmyadmin](#) - provides phpMyAdmin, a web-based interface to the MySQL database
- [traefik](#) - provides Traefik, a modern HTTP reverse proxy and load balancer that routes requests to the

appropriate container based on routing rules

Explore the full catalog of trusted content on Docker Hub:

- [Docker Official Images](#) – Curated images for popular software
- [Docker Hardened Images](#) – Security-hardened, minimal production images (also available at [dhi.io](#))
- [Docker Verified Publishers](#) – Images from verified software vendors
- [Docker Sponsored Open Source](#) – Images from sponsored OSS projects

Try it out

In this hands-on guide, you'll learn how to sign in to Docker Hub and push images to Docker Hub repository.

Sign in with your Docker account

To push images to Docker Hub, you will need to sign in with a Docker account.

1. Open the Docker Dashboard.
2. Select **Sign in** at the top-right corner.

3. If needed, create an account and then complete the sign-in flow.

Once you're done, you should see the **Sign in** button turn into a profile picture.

Create an image repository

Now that you have an account, you can create an image repository. Just as a Git repository holds source code, an image repository stores container images.

1. Go to [Docker Hub](#).
2. Select **Create repository**.
3. On the **Create repository** page, enter the following information:
 - **Repository name** - `getting-started-todo-app`
 - **Short description** - feel free to enter a description if you'd like
 - **Visibility** - select **Public** to allow others to pull your customized to-do app
4. Select **Create** to create the repository.

Build and push the image

Now that you have a repository, you are ready to build and push your image. An important note is that the image you are building extends the Node image, meaning you don't need to install or configure Node, yarn, etc. You can simply focus on what makes your application unique.

What is an image/Dockerfile?

Without going too deep yet, think of a container image as a single package that contains everything needed to run a process. In this case, it will contain a Node environment, the backend code, and the compiled React code.

Any machine that runs a container using the image, will then be able to run the application as it was built without needing anything else pre-installed on the machine.

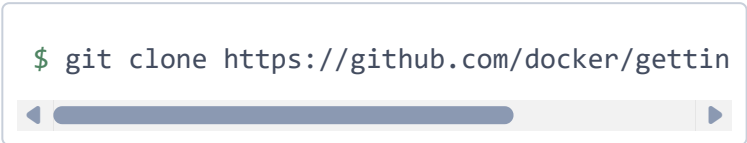
A `Dockerfile` is a text-based script that provides the instruction set on how to build the image. For this

quick start, the repository already contains the Dockerfile.

CLI VS Code

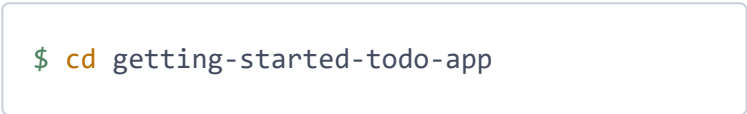
1. To get started, either clone or [download the project as a ZIP file](#) to your local machine.

```
$ git clone https://github.com/docker/gettin
```



And after the project is cloned, navigate into the new directory created by the clone:

```
$ cd getting-started-todo-app
```



2. Build the project by running the following command, swapping out `DOCKER_USERNAME` with your username.

```
$ docker build -t DOCKER_USERNAME/getting-st
```



For example, if your Docker username was `mobydock`, you would run the following:

```
$ docker build -t mobydock/getting-started-t
```

3. To verify the image exists locally, you can use the `docker image ls` command:

```
$ docker image ls
```

You will see output similar to the following:

REPOSITORY	TAG
mobydock/getting-started-todo-app	latest
...	

4. To push the image, use the `docker push` command.
Be sure to replace `DOCKER_USERNAME` with your username:

```
$ docker push DOCKER_USERNAME/getting-starte
```


Depending on your upload speeds, this may take a moment to push.

Recap

Before you move on, take a moment and reflect on what happened here. Within a few moments, you were able to build a container image that packages your application and push it to Docker Hub.

Going forward, you'll want to remember that:

- Docker Hub is the go-to registry for finding trusted content. Docker provides a collection of trusted content, composed of Docker Official Images, Docker Verified Publishers, and Docker Sponsored Open Source Software, to use directly or as bases for your own images.
- Docker Hub provides a marketplace to distribute your own applications. Anyone can create an account and distribute images. While you are publicly distributing the image you created, private repositories can ensure your images are accessible to only authorized users.

Usage of other registries

While Docker Hub is the default registry, registries are standardized and made interoperable through the [Open Container Initiative](#). This allows companies and organizations to run their own private registries. Quite often, trusted content is mirrored (or copied) from Docker Hub into these private registries.

Next steps

Now that you've built an image, it's time to discuss why you as a developer should learn more about Docker and how it will help you in your day-to-day tasks.

What's Next



Product
offerings

Pricing
About
us

llms.tx
llms-
full.txt

[Cookies Settings](#) | [Terms of Use](#) | [Status](#) | [Legal](#)

Copyright © 2013-2026 Docker Inc. All rights reserved.